

BIG CHALK

Mix Engine

User Guide

How to run a model, what every button does, and how to read what comes back.

Northwestern University Capstone · Summer 2026

Prepared for Alex Hathcock and Arkaparna Sen, Big Chalk Analytics

Version 2.0 — August 2026

Contents

1. Read this first
 2. Setting up and running the dashboard
 3. Anatomy of the screen
 4. Diagnostics — is this model any good?
 5. Variables — everything you can change
 6. Contributions — what drove the volume
 7. High Level — Total Brand and Total Channel
 8. Saturation — curves, correlations and ROI
 9. Multi-Level — unpooled, pooled, hierarchical
 10. Model Runs — running everything at once
 11. Export — getting it into Excel
 12. Definitions — the glossary inside the app
 13. How do I...? (recipes)
 14. Reading the numbers
 15. Files the tool reads and writes
 16. When something goes wrong
 17. How the modeling actually works
- Appendix A — What we found
- Appendix B — How this was verified

1. Read this first

The Mix Engine takes a workbook of weekly sales data and, for every Product × Channel combination in it, builds a regression that explains volume in terms of the things you can act on — distribution, price, promotion, media, competitors and the economy. It then tells you how much of your volume each of those accounted for, and how much of your year-over-year change each of them explains.

The point of the tool is not the individual model. It is that you can run seventy-six of them in about twenty seconds, see immediately which ones are wrong, and spend your time on those. Alex put it well on the 27 July call: "If it's easy, you don't really need a model. You draw a line and call it a day. It's always the weird stuff that you have to go look into — that's where your story is at."

The sixty-second version

- 1. Put your .xlsx path in the box at the bottom-left and click Load data.
- 2. Go to Model Runs and click Run all combinations. Wait ~20 seconds.
- 3. Sort the table by holdout MAPE. The bad models are at the bottom.
- 4. Click any row to open that model on Diagnostics.
- 5. Fix what needs fixing on Variables, hit Run, and look again.
- 6. Go to Export and download the Excel when you are happy.

What is new in this version

Three screens were added in August 2026, plus a set of controls on the existing ones. If you have used the tool before, these are the parts you have not seen:

Screen	What it gives you
High Level	Rolls every Product × Channel model up to Total Brand or Total Channel, so you can see the overall fit and contributions instead of one slice at a time.
Saturation	The weekly picture of what the engine actually fed the regression — raw execution, decayed execution, and decayed + saturated — with each one's correlation to the target. Plus the execution-vs-ROI curve and the optimal spend point. Sliders let you override the decay, midpoint and slope and watch everything redraw.
Multi-Level	The same product modelled three ways — one model per channel (unpooled), one model for the whole product (pooled), and a hierarchy where each channel inherits the product-level coefficient with its own adjustment — all scored on the same holdout so you can see which wins.

NO REGRESSIONS Nothing about the existing screens changed. Every model built before this release produces exactly the same numbers.

2. Setting up and running the dashboard

One-time setup

You need Python 3.10 or later. From a terminal, in the project folder:

```
pip install -r requirements.txt
```

That installs numpy, pandas, scipy, statsmodels, matplotlib, openpyxl, dash and plotly. It takes a couple of minutes the first time and is never needed again.

Starting it

```
cd code  
python dashboard.py
```

The terminal prints a line like

wiring OK: 111 components, 42 callbacks and then a local address. Open <http://127.0.0.1:8050> in a browser. Leave the terminal window open — closing it stops the app.

WHY THAT LINE The "wiring OK" line is a self-check that runs at startup. It walks the whole screen layout and confirms every control a callback refers to actually exists. If a control is ever renamed and something is missed, the app refuses to start and says which one, instead of loading fine and silently ignoring that button.

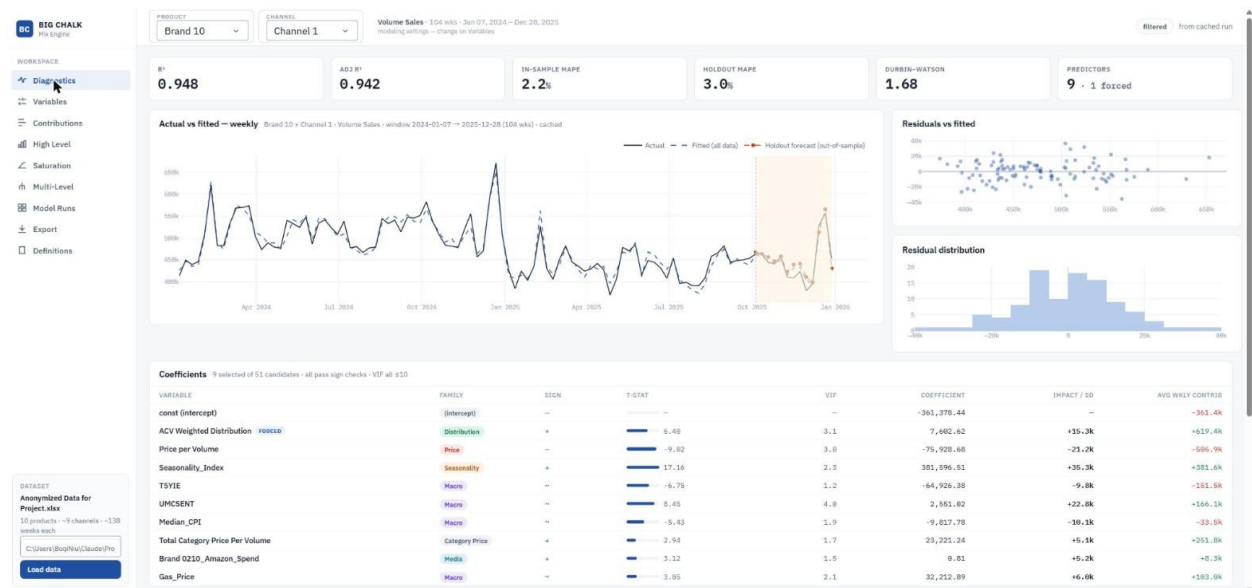
Loading your data

Bottom-left of the screen, under DATASET, paste the full path to your .xlsx and click Load data. The tool then reads:

- one worksheet per product (a sheet called "General Data Dictionary" is skipped automatically);
- a Geography column, which is the channel;
- a Time column, which is the week;
- every numeric column as a candidate predictor.

You do not have to tell it which columns are media or price — it works that out from the names and writes a starting configuration you can then edit. See §5.

3. Anatomy of the screen



The workspace: nine screens in the left rail, the results filter and read-only settings note along the top, the loaded dataset bottom-left.

Part of the screen	What it is
Left rail — WORKSPACE	The nine screens. Clicking one switches the main panel; nothing is recalculated, so switching is instant.
Left rail — DATASET	Which workbook is loaded, and the box to load a different one.
Top bar — PRODUCT / CHANNEL	A results filter and nothing more. Changing it shows you a different model that has already been built; it does not re-run anything.
Top bar — settings note	Read-only echo of the current target variable, window length and the exact calendar dates being modelled.
Top bar — run status	When the last run happened and how long it took.

WHY THE SPLIT PRODUCT and CHANNEL filter results. TARGET and WINDOW are modelling decisions — changing them means re-running — so they live in one place, on the Variables screen, and are only echoed at the top. This was Alex's instruction on 27 July: "the only thing you should be filtering on are results, which would be your product and channel."

The one thing to understand about how it runs

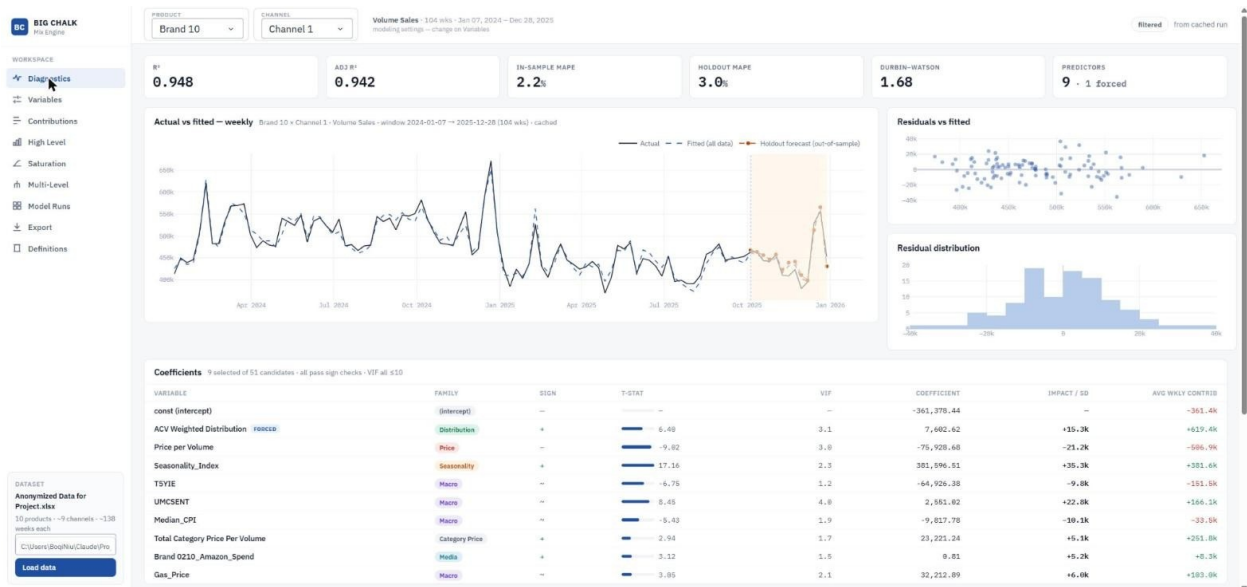
Models are built once and cached. When you click Run all combinations, every Product × Channel is fitted and the full results — not just a summary — are held in memory. Every other screen then filters that cache. Switching product, changing the contributions period, opening High Level: all of these read the cache in about a tenth of a second and none of them refit anything.

This matters for a reason beyond speed. If each screen refitted on demand, two screens could quietly show you numbers from two different fits. They cannot, because there is only ever one set of results.

The cache is keyed by the datafile, the target, the window length and the selection settings. Change any of those and you get a fresh cache rather than stale numbers from the previous configuration.

4. Diagnostics — is this model any good?

The single-model view. Everything here is about the Product × Channel selected in the top bar.



Diagnostics — Brand 10 x Channel 1. R² 0.948, holdout MAPE 3.0%. The shaded band on the right of the chart is the reserved holdout; the orange line is the forecast into it.

The metric strip

Metric	What it means and what to worry about
R ²	Share of the week-to-week variation the model explains. Higher is better, but a high R ² on an in-sample fit proves very little on its own — look at holdout MAPE.
Adjusted R ²	The same, penalised for the number of predictors. If this is much lower than R ² , the model is carrying variables that are not earning their place.
MAPE (in-sample)	Average weekly percentage error on the weeks the model was fitted to. Optimistic by construction.
Holdout MAPE	THE number. The last 13 weeks are held back, the variable selection never sees them, and this is the error on those weeks. Under ~15% is good, over 40% is flagged.
Durbin-Watson	Whether the residuals are autocorrelated. Near 2 is fine. Near 0 means the model is systematically missing a pattern over time — usually a trend or a seasonal effect that is not in the model.
REVIEW flags	Sign conflicts (a coefficient the data pushes the opposite way from your prior) and high-VIF variables. Neither blocks anything; both are worth looking at.

Actual vs fitted

The black line is what happened, the blue dotted line is what the model says should have happened, and the shaded band on the right is the holdout. Look at the shaded band first: if the two lines separate there but track well everywhere else, the model has learned the past and cannot predict.

The coefficient table

Column	What it is
--------	------------

Variable	The predictor, with a coloured chip for its family.
Coefficient	Volume per one unit of this variable, in whatever units the variable is in.
Impact / SD	Volume per one standard-deviation move. This is the column to rank drivers by. Raw coefficients cannot be compared across variables because the variables live on completely different scales — Arko's point on 27 July, and the reason this column exists.
t	How well the data identifies the coefficient. $ t $ under about 2 means it is not distinguishable from zero.
VIF	Collinearity with the other predictors. Over 10 is flagged. High VIF does not make the model wrong, but it makes the individual coefficients unstable and hard to defend.
Avg weekly due-to	The average weekly volume attributable to this variable. For media, averaged only over weeks with actual execution — never over the zeros of a flight that ran for six weeks.
Forced	Marked if you pinned this variable in rather than letting selection choose it.

WORKED EXAMPLE An "Impact / SD" that is large while the coefficient looks small is completely normal and is the point of the column. On Brand 1 × Channel 1, seasonality and ACV distribution have raw coefficients of 61,473 and 3,107 — twenty times apart — but impacts of 13,193 and 13,151, which is a dead heat.

5. Variables — everything you can change

This is the control room. Every modelling decision lives here, and this is the only screen with a Run button that rebuilds the current slice.

PRODUCT
Brand 10

CHANNEL
Channel 1

Volume Sales - 104 wks - Jan 07, 2024 - Dec 26, 2025
modelling settings - change on Variables

Filtered - from cached run

MODELING SETTINGS
These define the model itself. Changing them requires a re-run — that is why they live here and not on results screen.

TARGET
Volume Sales

WINDOW
104 wks

Window: 104 wks - Jan 07, 2024 - Dec 26, 2025 (same range for every Product × Channel)

VARIABLE SELECTION
How hard the model is to get into.

STRICTNESS
Balanced (p<0.05, VIP 10)

p<0.05 - VIP<10 pruned - no family caps - re-run to apply

Max variables per family

FAMILY

Distribution

Trade

Media

Competitive

Macro

Price

Category Price

Seasonality

Trend

Blank = no limit. Set a number to cap how many of that family may enter — e.g. Trade 3, Competitive 2, Macro 2, Media left blank is Alex's "only two to three trade, but all media".

The cap is applied while selecting, not by trimming afterwards, so a family spends its budget on its most significant members and the other families still fill up normally.

Forced variables are always kept and count toward the budget — a cap of 2 with 2 forced Trade variables admits no more Trade.

CONFIG

Product default (all channels)

DUE-TO PERIOD

Total

☐ Hide excluded

Upload template

Download template

Reset to default

Regenerate defaults

Save config

Optimize media curves

Run model

Editing Brand 10 Product default — used by every channel that has no override.

Sign is a guardrail, not a wish. Positive-signed variables can never enter with a negative coefficient. Role force keeps a variable in every run; exclude bans it. Custom bounds override the sign. Adstock applies to media only.

VARIABLE	FAMILY	EXPECTED SIGN	ADSTOCK	SCALE	SAT. RES	SAT. SLOPE	BOUND LO	BOUND HI	CHEF. WOK	IMPACT / SD	CONTRIBUTION	ROLE
Price per Volume	Price	negative	-	1					-75,929	-21.2k	-52.7%	auto
Total Category Price Per Volume	Category Price	positive	+	1					23,221	+5.1k	+26.2%	auto
APV Weighted Distribution	Distribution	noctive	-	1					9,403	+16.1%	+24.4%	force

Variables — modelling settings at the top, strictness and the family-cap table in the middle, the per-variable grid below. The banner states which config scope is in force.

Modelling settings

Control	What it does
Target	Which column is being predicted. Usually Volume Sales; any numeric column can be chosen. Changing it invalidates the cache and requires a re-run.
Window	52, 104 or 156 weeks. The window always ENDS on the latest week present anywhere in the workbook and counts back — the same calendar dates for every slice, no exceptions.
Run	Rebuilds the current Product × Channel with whatever is on this screen and jumps you to Diagnostics.

WHY THE WINDOW IS FIXED The fixed window is not a cosmetic choice. Before it, each slice used the last N rows it happened to have, so a channel that stopped selling in mid-2023 still produced a model — from 2023 data — and leaked that disjoint chunk into the weekly export. Alex found it by pivoting the export in Excel. The window is now an absolute date range, identical everywhere, and a slice without enough data inside it is not modelled at all; it appears in a "Not modelled" table with the reason.

The variable table

One row per candidate predictor. Edit any cell and hit Run.

Column	What it does
Family	Grouping — Distribution, Trade, Media, Competitive, Macro, Price, Seasonality, Trend. Drives the

	colour chips and the per-family caps.
Sign	positive, negative or unconstrained. A sign constraint is enforced during fitting, not checked afterwards — a media variable set to positive cannot come back negative.
Adstock decay	Carry-over. 0 means all of this week's execution lands this week; 0.8 means most of it decays in over following weeks. Only meaningful for media.
Scale	Divide the variable by this before modelling, so a coefficient reads "per 1,000 impressions" instead of "per impression". Purely a units change — every contribution and fitted value is identical to the last decimal place.
Sat. midpoint (y)	Half-saturation point, as a fraction of the peak. At this execution level the variable delivers half of everything it can deliver. Blank = no saturation.
Sat. slope (s)	Curve shape. Above 1 gives an S-curve (slow start, steep middle, flattening); below 1 gives a curve that diminishes straight from the origin. Blank = no saturation.
Coef lower / upper	Hard bounds on the coefficient, in the variable's original units. Overrides the sign setting.
Role	auto lets selection decide; force pins the variable in whether it is significant or not; exclude removes it from consideration entirely.

NOTE Both saturation parameters must be filled in, or neither. Half a specification is treated as "no saturation" rather than guessed at.

Two-tier configuration: product default and channel override

Every product has a default configuration that applies to all its channels. You can also save a configuration for one specific Product × Channel, which wins wherever it exists. The scope selector at the top of the table controls which one you are editing, and the screen tells you which is currently in force.

Button	What it does
Save config	Writes the table to whichever scope is selected.
Reset to default	Deletes the channel override so the product default applies again.
Regenerate	Throws the configuration away and rebuilds it from the data using the same heuristics as a first load.
Download template	Exports every product's configuration as one CSV.
Upload template	Reads that CSV back. This is the bulk-edit path — set up fifty variables in Excel rather than fifty clicks. Writes are atomic: either the whole upload applies or nothing does.

Selection strictness and family caps

Alex asked on 27 July whether the selection could be made deliberately more or less permissive. It can:

Preset	Settings	Typical result
Punitive	$p < 0.01$, VIF > 5 pruned	≈ 4.4 variables per model
Balanced	$p < 0.05$, VIF > 10 pruned	≈ 6.5 — the default
Lax	$p < 0.15$, VIF > 20 pruned	≈ 10.0

Underneath is a per-family cap table — "I want only two to three trade variables, but I want all media". Blank means no limit. The cap is enforced while selection runs, not by trimming afterwards, so a capped family spends its budget on its most significant members and the other families are still selected properly. Forced variables are always kept and do count against their family's budget.

WORTH TRYING Caps default to blank, so out of the box every model is identical to before this control existed. Worth knowing: on Brand 1 × Channel 1, capping Trade at 1 and Macro at 1 IMPROVED holdout MAPE from 23.0% to 21.5%. Fewer variables is sometimes simply better.

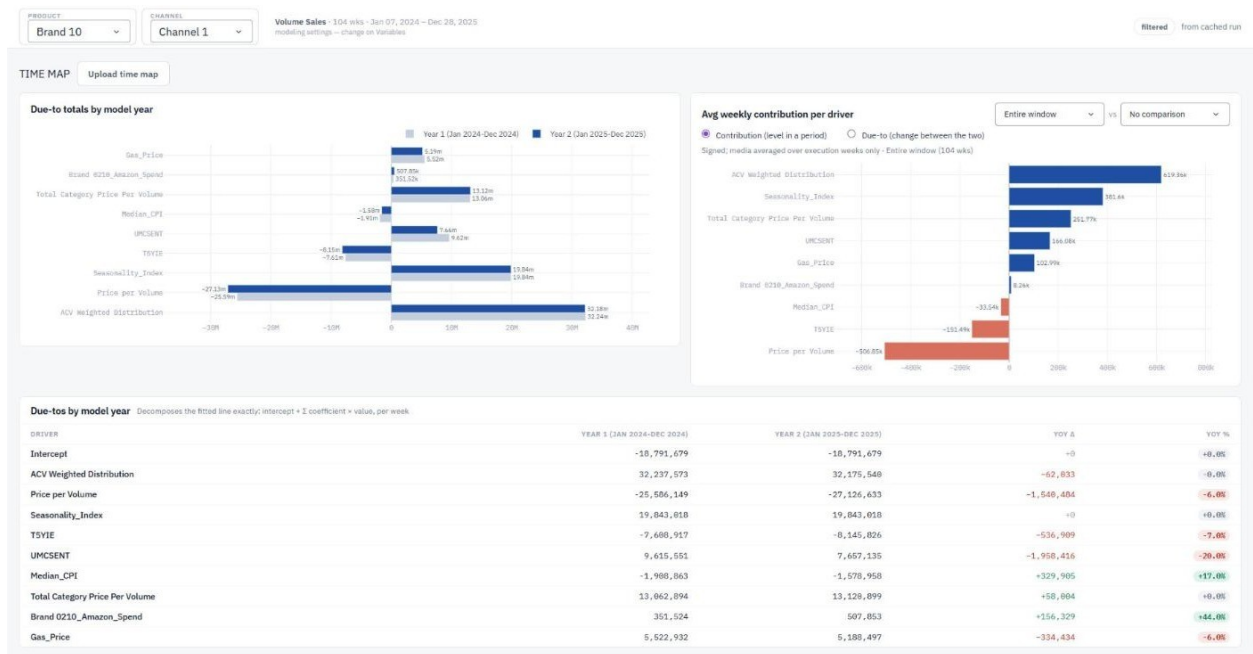
Optimize media curves

This button searches, for every media variable, over decay × midpoint × slope — including the option of no saturation at all — and proposes the combination that scores best. It is scored by rolling-origin cross-validation on inner folds, with the reported holdout kept completely out of the search, so the holdout MAPE you see afterwards is still an honest out-of-sample number.

It writes candidates into the configuration and does not run anything. Look at them, adjust them on the Saturation screen if you disagree, then hit Run.

NOTE Read Appendix A before you trust this button. Across 25 slices it improved its own cross-validation score every time it fired, but improved the untouched holdout only half the time. It ships switched off for that reason.

6. Contributions — what drove the volume



Contributions — due-to totals by model year on the left, average weekly contribution per driver on the right, and the year-over-year table underneath.

Contribution is not the same as due-to

These are two different numbers and the distinction is Arko's, from 27 July:

Term	Definition	The question it answers
Contribution	A level: the volume this driver accounts for during a period.	"How much of my volume is media?"
Due-to	A change: contribution in period B minus contribution in period A.	"Why am I down year on year?"

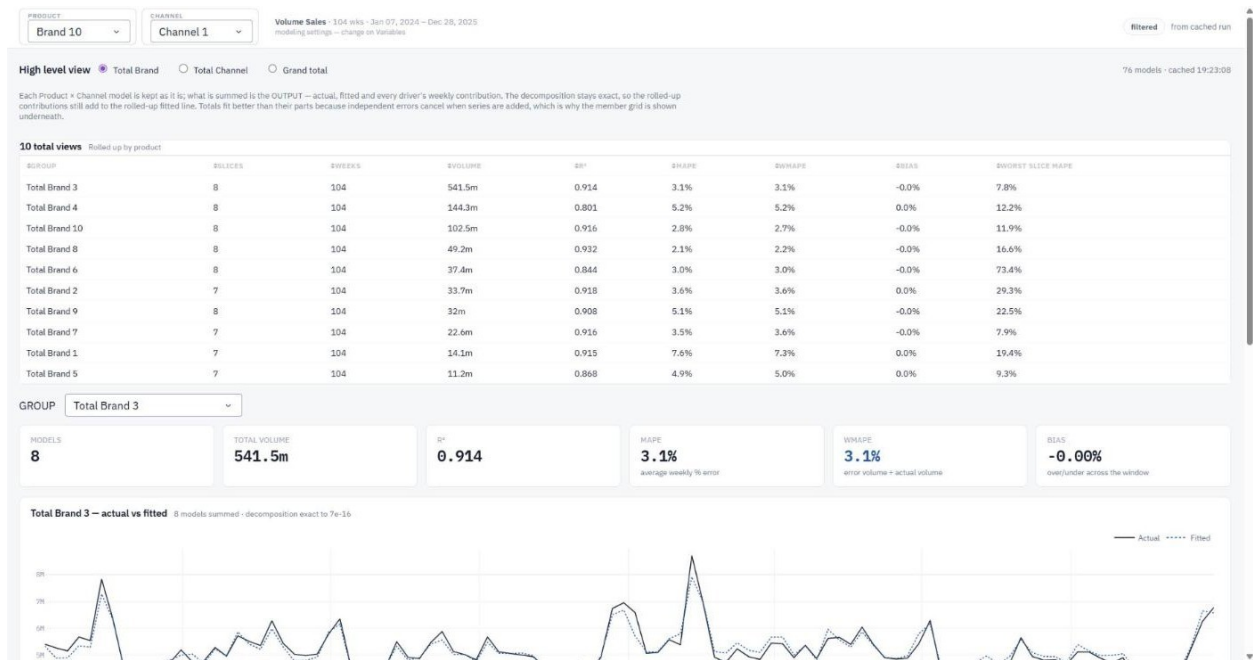
Both are available and the toggle at the top switches between them. The due-to is what most readers of a deck actually want. Period totals alone cannot answer "how is the change in macro variables affecting my sales year to year", because that is a question about a difference.

Controls

Control	What it does
Period	Model year 1, model year 2, the latest quarter, or any period you have uploaded a mapping for.
Compare to	The baseline period for a due-to. Periods of unequal length are compared per week automatically, so a 52-week period does not look bigger than a 13-week one just for being longer.
Upload period map	A CSV of week → period label, for fiscal calendars that do not line up with the tool's default 52-week blocks.
Hide excluded	Drops variables with zero contribution from the chart.

7. High Level — Total Brand and Total Channel

New in this release. Every screen so far shows one Product × Channel at a time. This one adds them up.

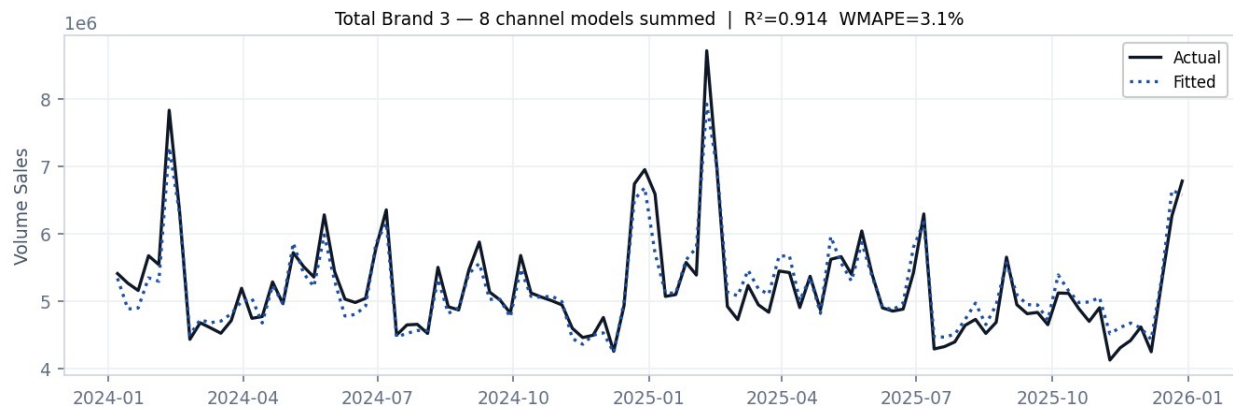


High Level — all ten Total Brand views, with Total Brand 3 opened below: eight channel models summed, R² 0.914, WMAPE 3.1%.

What it does

Pick a rollup mode and the tool sums the OUTPUT of the individual models — actual volume, fitted volume, and every driver's weekly contribution — across the group.

Mode	What gets added together
Total Brand	For each product, all of its channels. "How is Brand 3 doing overall?"
Total Channel	For each channel, all of the products in it. "How is Channel 3 doing across the portfolio?"
Grand total	Everything, in one line.



Total Brand 3 — eight channel models summed. The decomposition stays exact under summation, so the rolled-up contributions still add to the rolled-up fitted line.

What it is NOT

No new model is fitted here. Each Product × Channel keeps its own coefficients; only the results are added. That is a real distinction, and it changes what the fit statistics mean.

NOTE A rolled-up R^2 and MAPE will always flatter the portfolio, because independent errors partly cancel when you add series together. Total Brand 3 shows R^2 0.914 and WMAPE 3.1%, while its worst individual channel is at 7.8% MAPE. That is a genuine property of the aggregate, not a trick — but it is exactly why the member grid sits underneath every total. A good total can still hide a bad slice, and the grid is there so it cannot hide it from you.

If what you actually want is one model for the whole brand — one coefficient per predictor, fitted across every channel at once — that is the pooled model on the Multi-Level screen, and it is a different thing.

What is on the screen

Element	What it tells you
Summary table	One row per total group: number of models, volume, R^2 , MAPE, WMAPE, bias, and the MAPE of the worst slice inside it.
Group selector	Pick one total to open in detail.
Metric strip	The six headline numbers for the selected total.
Actual vs fitted	Weekly, at the total level.
Total contributions	Each driver's contribution over the whole window and its share of modelled volume.
Year-over-year due-to	The latest 52 weeks against the 52 before, by driver — what changed and how much of the change each driver explains.
Models in this total	The parts the total is made of, with each one's own fit statistics.

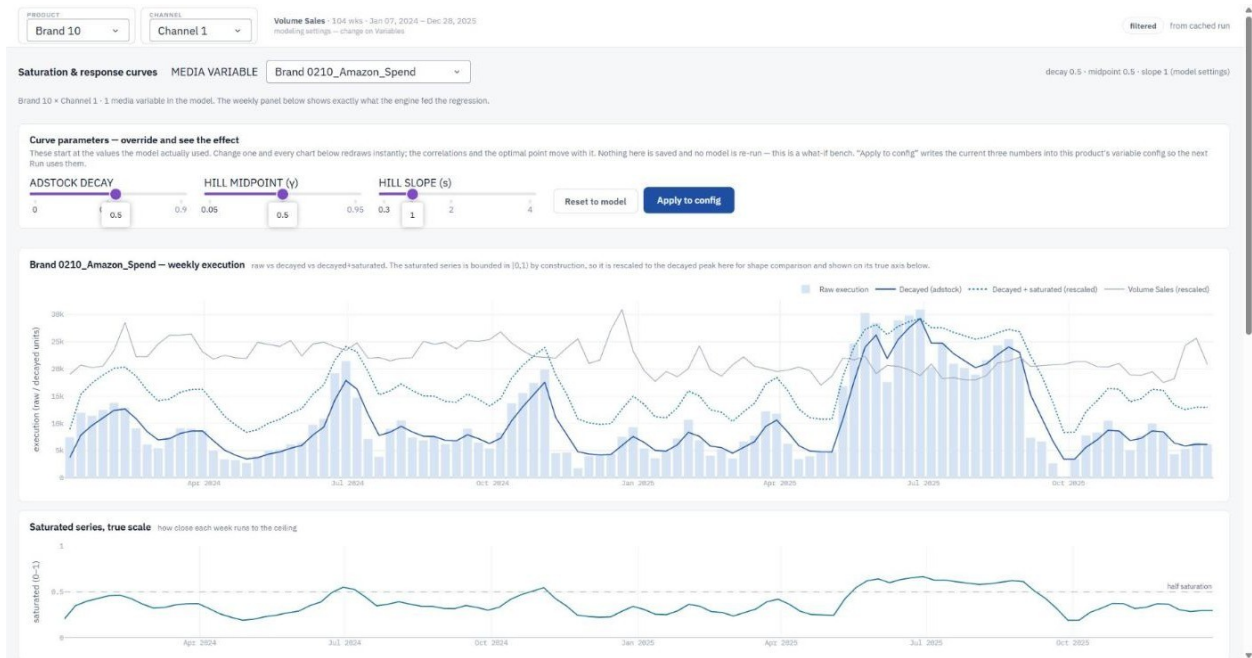
WHICH METRIC WMAPE (error volume ÷ actual volume) is the metric to quote at the total level. Plain MAPE averages week-level percentages, which lets a small week count as much as a large one. WMAPE weights by the volume actually at stake, which is what "how far off is my volume" means.

The coverage warning

If some slices do not cover the whole window — a product that launched in a channel part-way through, for instance — the total will dip at the left edge for a reason that has nothing to do with performance. The screen says so explicitly when it happens, naming how many weeks are affected. On the current dataset this fires for Total Brand 9, where one channel covers 67 of the 104 weeks.

8. Saturation — curves, correlations and ROI

New in this release. Two things live here: a weekly diagnostic that shows exactly what the engine fed the regression, and an annual ROI curve that says whether you are spending too much or too little.



Saturation — the three override sliders, the weekly raw/decayed/saturated panel with the target overlaid, and the saturated series on its own 0-1 axis below.

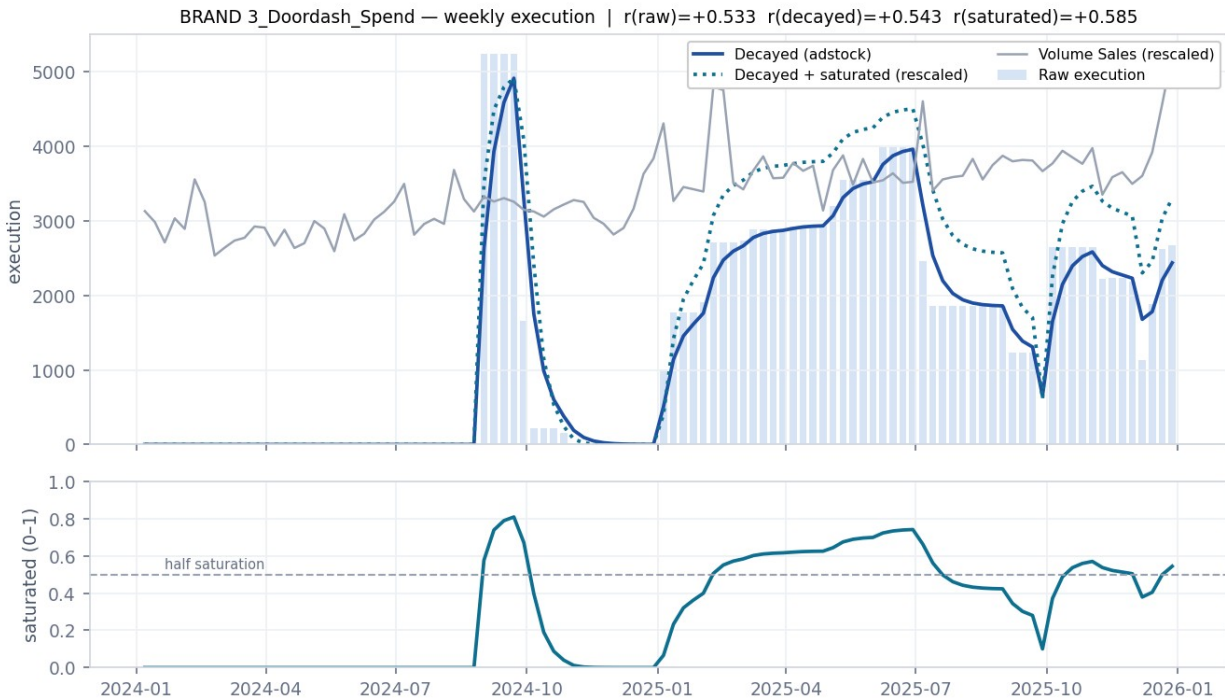
The parameter sliders

This is what Alex asked for on the 11 August call: "I want something that can go — okay, what if I want to override this visually? Here's the graph, here's the decayed media, I'm going to change from a 0.2 to a 0.3. What does that look like? How much worse is it?"

Control	What it does
Adstock decay	How much of this week's execution carries into following weeks.
Hill midpoint (γ)	The half-saturation point, as a fraction of peak execution.
Hill slope (s)	The curve shape. Above 1 is an S-curve; below 1 diminishes from the origin.
Reset to model	Returns all three sliders to the values the model actually used.
Apply to config	Writes the three numbers into this product's variable configuration so the next Run uses them. Saves only — it does not run anything.

WHAT THE SLIDERS DO AND DO NOT DO Dragging a slider redraws every chart from the cached slice. No model is re-run: the coefficient is held at its fitted value while the shape of the input changes. So what you are watching is the transform and its correlation with the target, not a refitted model. The screen says so, because pretending otherwise would be a lie. To find out what a setting does to holdout error, apply it and hit Run.

The weekly panel



Raw execution (bars), decayed execution after adstock (solid), decayed + saturated after Hill (dotted), and the target overlaid — with the correlation of each transform to the target in the title. Below, the saturated series on its own true 0–1 axis.

Alex flagged the scale problem in the brief: the saturated series is bounded in $[0, 1]$ by construction, while raw and decayed are in dollars or impressions. On a shared axis the saturated line would be a flat smear along the bottom. So it appears twice — rescaled to the decayed peak so its SHAPE can be compared, and again below on its own axis so its LEVEL can be read. The dashed line there is half saturation: weeks above it are past the halfway point of everything that medium can deliver.

The correlations are computed on the unscaled series. A positive linear rescale cannot change a Pearson correlation, so the two agree — the rescale is presentation only.

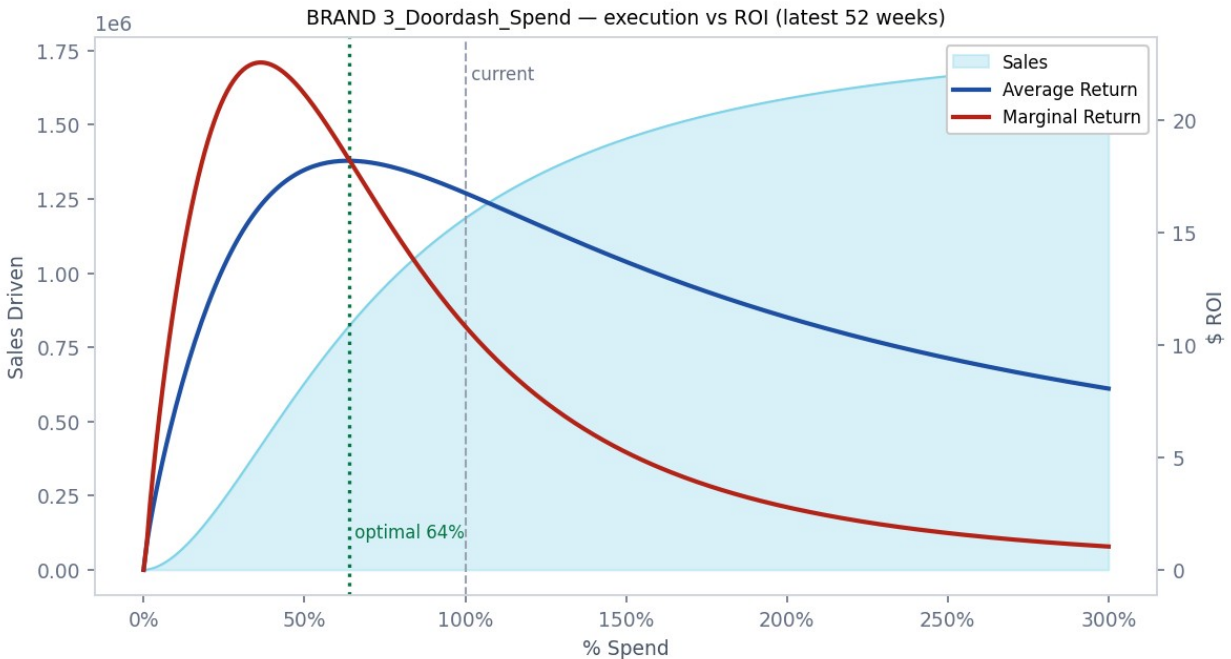
The correlation table

Three rows — raw, decayed, saturated — each showing the correlation with the target under the model's own settings and under yours, and the change. This is the evidence behind a decay or midpoint choice, which is what Alex meant by "I want to see the reasoning behind it".

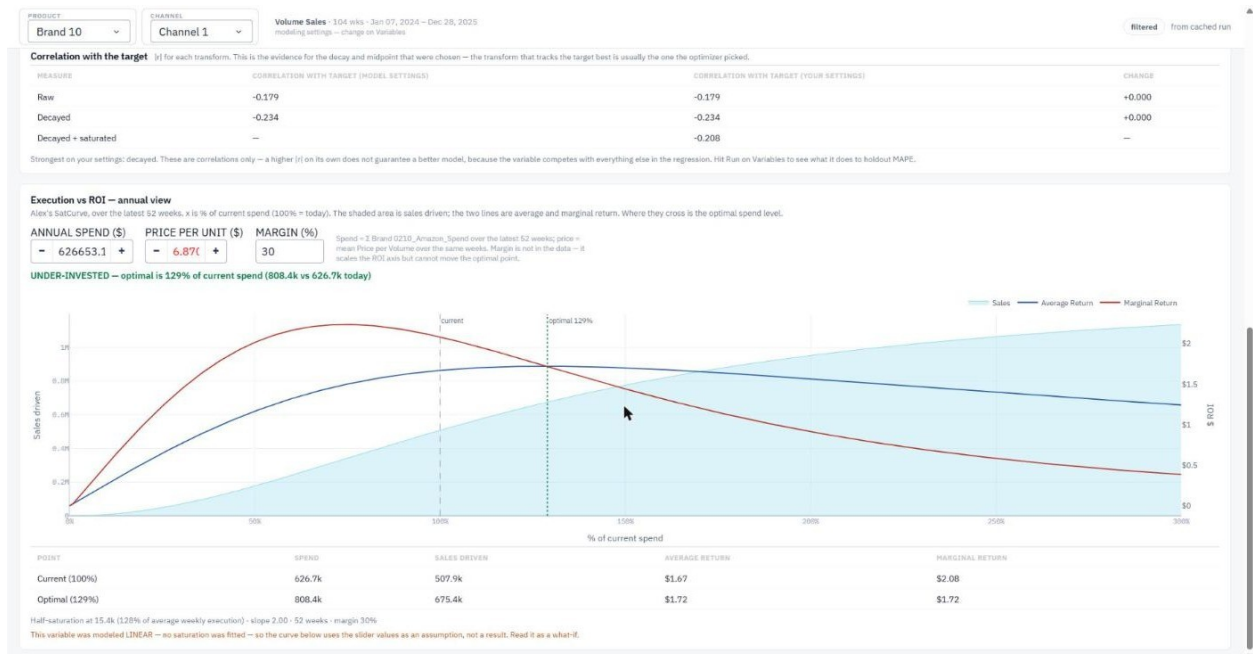
NOTE A higher correlation does not by itself guarantee a better model. The variable competes with everything else in the regression, and a transform that correlates better in isolation can still lose to another predictor once both are in. The table is evidence, not a verdict.

The execution-vs-ROI curve

This is a port of Alex's own SatCurve class, matching it line for line so the numbers agree with what he gets from his own code. It takes the latest 52 weeks and asks what would happen if the whole year's spend were scaled up or down together.



x is % of current spend (100% = today). The shaded area is sales driven, read on the left axis. The two lines are average and marginal return on the right. Where they cross is the optimal spend level.



The live screen further down: the three correlations, then the ROI curve. Here the slope slider has been raised to 2, which turns the curve S-shaped and produces a crossover at 129% of current spend on real data.

The reading is: past the crossover, the next dollar returns less than the average dollar you have already spent. In the example above the crossover is at 53% of current spend, so this variable is over-invested — the model says roughly half the current budget is doing most of the work.

Input	Where it comes from	Editable?
Annual spend	Sum of the media column over the latest 52 weeks — in this dataset the media columns ARE spend.	Yes
Price per unit	Mean Price per Volume over the same weeks.	Yes
Margin	Not present in the data anywhere. Defaults to 30%.	Yes
Sales driven	The model's contribution for this variable over those weeks. Anchors the curve at the current point.	No — it is the model result

ABOUT MARGIN Margin moves the ROI axis but CANNOT move the optimal point. It scales average and marginal return by exactly the same factor, so the crossing stays where it is. Worth knowing before anyone spends an afternoon arguing about the right margin assumption. This is asserted in the test suite, not just claimed here.

When the tool refuses to give you an optimal point

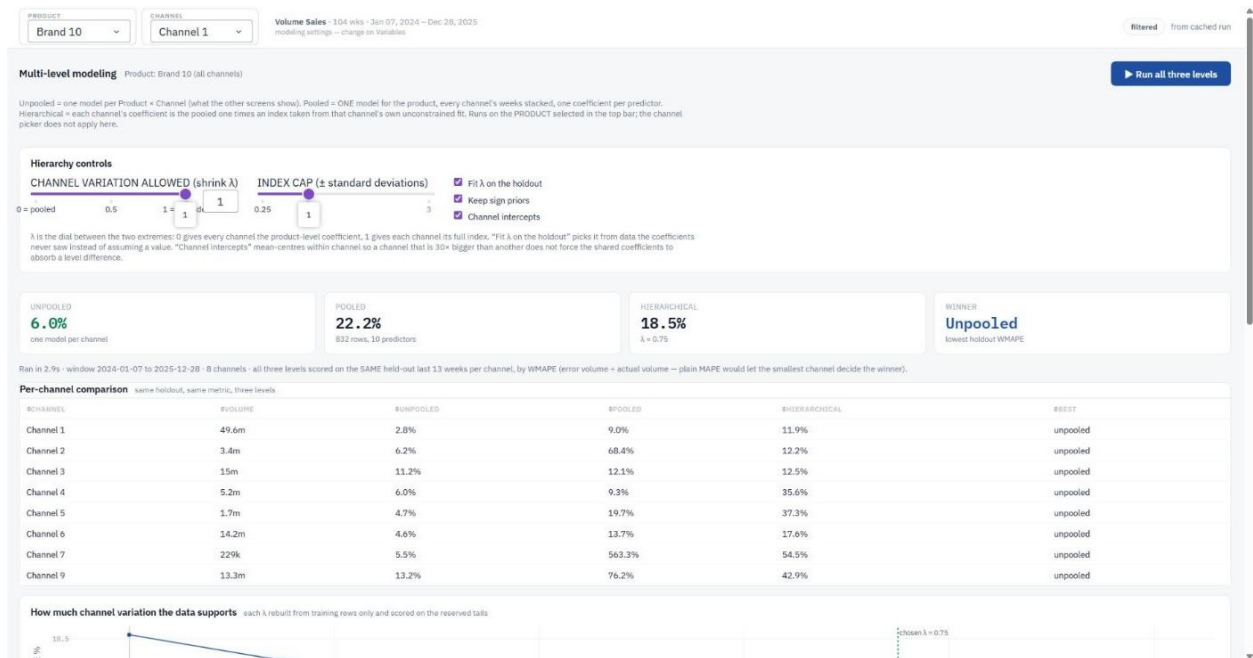
With a slope below 1 the curve is concave from the origin. In that case marginal return sits below average return at every single spend level and the two lines never cross — this is provable from the Hill equation, not a quirk of one dataset. The search would otherwise return the last point on the grid, and the screen would show "optimal = 300%", which would be a graphing artifact presented as a recommendation to triple your budget.

So instead the screen reports it as inconclusive and says why. If you believe the medium has a genuine S-curve, raise the slope above 1 and the crossover appears.

NOTE If a media variable was modelled linear — which is the default, since saturation ships off — there is no fitted curve to draw. The screen still gives you one, using the slider values as an assumption, and labels it clearly as a what-if rather than a result.

9. Multi-Level — unpooled, pooled, hierarchical

New in this release. The same product modelled three ways, all scored on the same holdout.



Multi-Level — Brand 10. Unpooled 6.0%, pooled 22.2%, hierarchical 18.5% at the fitted lambda of 0.75, all on the same holdout. The winning level is called out.

The three levels

Level	What it is	The trade-off
Unpooled	One independent model per Product $\times$ Channel — what every other screen shows.	Most flexible; only ~104 weeks per model, so coefficients can differ between channels for no reason but noise.
Pooled	ONE model for the product, every channel's weeks stacked. One coefficient per predictor, shared.	700–900 rows against the same predictor count — far more degrees of freedom, but individual channels lose their own responsiveness.
Hierarchical	Each channel's coefficient = the pooled coefficient $\times$ an index from that channel's own fit.	A middle ground: channels differ, but only as much as the data supports.

Arko framed it in market-mix terms on the call — pooled shares one coefficient across markets, unpooled gives each market its own, and partially pooled sits between them with "some kind of relationship between the national coefficient and the market-level coefficient". Here the markets are channels and the top level is the product.

Why stacking channels is legitimate

Alex made this argument on the call and the whole design rests on it, so it is worth stating plainly: these are models OF a time series, not time-series models. There is no term relating this week to last week — adstock and

saturation exist precisely to turn carry-over into a property of each row. Once that is done the rows are independent, and independent rows can be stacked.

The engine applies adstock and saturation WITHIN each channel, on that channel's own contiguous weekly series, and only then stacks. Doing it the other way round would carry Channel 1's December spend into Channel 2's January.

National predictors

Before pooling, any predictor that carries the same values in every channel — unemployment, CPI, most media — has to be split up. Otherwise one shared coefficient produces that contribution once per channel: with nine channels, nine times the effect the variable can possibly have had. Each is multiplied by its channel's share of product volume, so the per-channel pieces sum back to the national figure exactly. The screen shows which predictors were treated this way and reports the conservation check.

A REAL FALSE POSITIVE Alex suggested a quick screen for this: sum each predictor by channel, and if the sums match it is national. On this data that flags 26 variables — and one of them is not national at all. Seasonality_Index is a per-channel index normalised to average 1.0, so every channel sums to exactly 104.000000 over a 104-week window while the weekly values differ a great deal (2024-01-07: 0.947 in Channel 1 against 0.565 in Channel 9). Equal sums, different data. Splitting it would have quietly destroyed a strong channel-specific predictor. The engine therefore uses Alex's DEFINITION — "the exact same values for each brand*channel" — and compares the weekly values, while still running the sum screen and listing any near-miss on screen.

Channel intercepts — the setting that matters most

Arko said "we should do the full modelling, or like the mean centred models". This is that, and it is the difference between a pooled model that works and one that cannot.

Within Brand 1 the channels differ in volume by thirty times — Channel 1 is about 7.1M over the window, Channel 9 about 0.22M. A single shared intercept has to sit somewhere between them, so the big channel is under-predicted and the small one over-predicted by a large constant every week, no matter how good the slopes are. The model then burns its coefficients trying to explain a level difference that has nothing to do with the drivers.

With the box ticked, each channel gets its own base level via mean-centring within channel. What stays pooled is what Alex wanted pooled — one coefficient per predictor. What varies is only the base level, which is a fact about channel size.

HOW MUCH IT MATTERS This one setting took Brand 3's pooled holdout error from 39.2% to 13.6%, and its hierarchical from 36.9% to 7.2%. Before it, the hierarchical model looked broken on every product; after it, it wins outright on three of ten. The instability was a specification problem, not a flaw in Alex's method.

How the hierarchy is built

Three runs, in the order Alex laid out in his email:

1. Pooled, with sign constraints, on the stacked product. Its selected structure becomes the shared predictor set, and its coefficients are the fixed effect.

2. One unconstrained fit per channel on that same predictor set — no priors, no sign control. This is deliberate: a constrained fit piles coefficients onto the boundary at exactly zero, and a coefficient pinned at a bound carries no information about how that channel differs. The random effect has to come from an unconstrained fit.
3. Each channel's final coefficient = its index $\times$ the pooled coefficient, with the index optionally capped and shrunk, and the intercept re-solved on that channel.

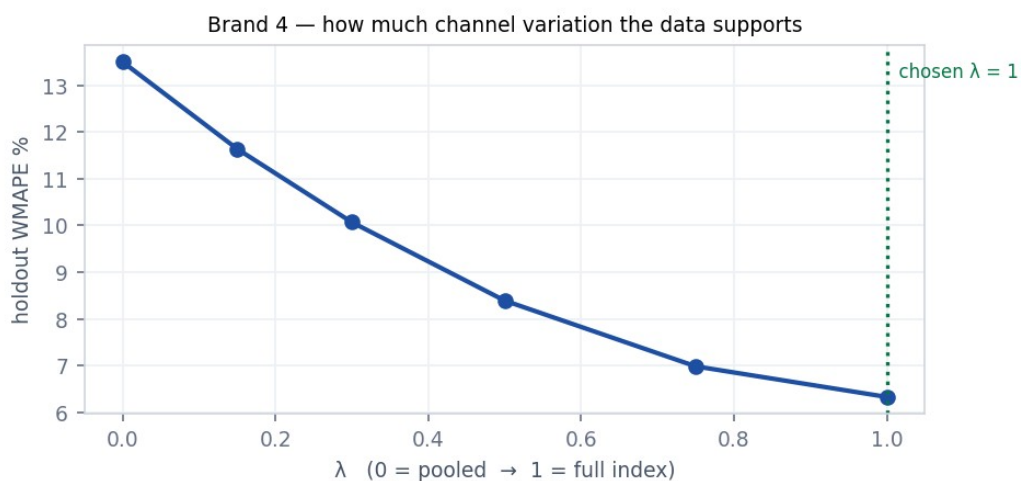
The index is that channel's unconstrained coefficient divided by the mean across channels, so the indices average to exactly 1 and the final coefficients average back to the pooled ones. That is the check in rows 20–25 of Alex's spreadsheet, and it is asserted in the test suite.

The controls

Control	What it does
Channel variation (λ)	The dial between the two extremes. 0 gives every channel the product-level coefficient (= pooled); 1 gives each channel its full index (= Alex's method). In between is literal partial pooling.
Index cap	Clips each index to $1 \pm k$ standard deviations, from Alex's sheet. This is what keeps it partial pooling rather than a relabelled unpooled model — a channel whose own fit produced a wild coefficient is pulled back toward the product effect.
Fit λ on the holdout	Chooses λ from data the coefficients never saw instead of assuming a value. On by default.
Keep sign priors	Clips the index at 0 so a channel's coefficient can never flip sign against the pooled one. Leave this on.
Channel intercepts	Mean-centring within channel, as above. Leave this on.
Run all three levels	Fits everything and fills the screen. Takes a few seconds per product.

WHY THE SIGN GUARD EXISTS "Keep sign priors" is not in Alex's spreadsheet — it was added because on real data the raw index runs from -7.2 to $+11.9$. A negative index flips the sign of the final coefficient, so a channel ends up with media that destroys sales and a price cut that reduces volume, silently undoing the sign constraints set in the variable configuration. The sheet's example coefficients are far better behaved than real ones.

Reading the λ curve



Brand 4. Each λ is rebuilt from training rows only and scored on the reserved tails, so the curve is out-of-sample throughout. Here more channel variation keeps helping all the way to $\lambda = 1$.

The shape is the interesting part. A curve that rises from left to right means the channels genuinely do not differ and the pooled model is the honest answer. A curve that falls, as above, means they do. A flat curve means the question does not matter for that product.

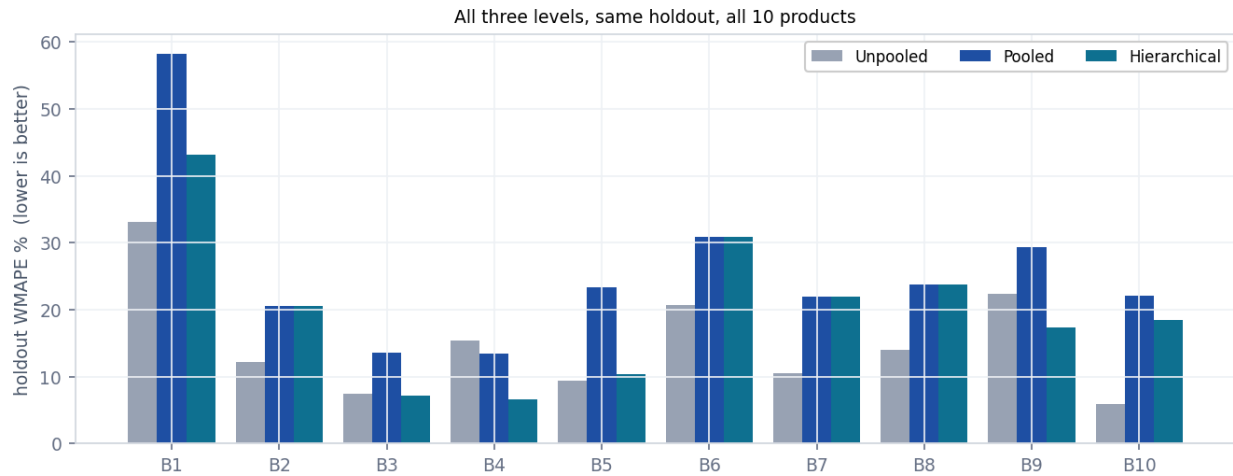


Lower on the same screen: the lambda curve, the national predictors that were split (with the Seasonality_Index near-miss flagged), the per-channel coefficient index, and how well the pooled coefficients serve each channel.

The unstable-index warning

The index divides by the mean coefficient across channels. When that mean is small relative to its spread — when the channels cannot even agree on the sign — the ratio is unstable and the index means very little. The screen reports how many predictors are in that state. On Brand 1 it is fifteen of sixteen, which tells you immediately why that product does not benefit from a hierarchy. Those are exactly the rows the cap and λ exist to tame.

Which level should you use?



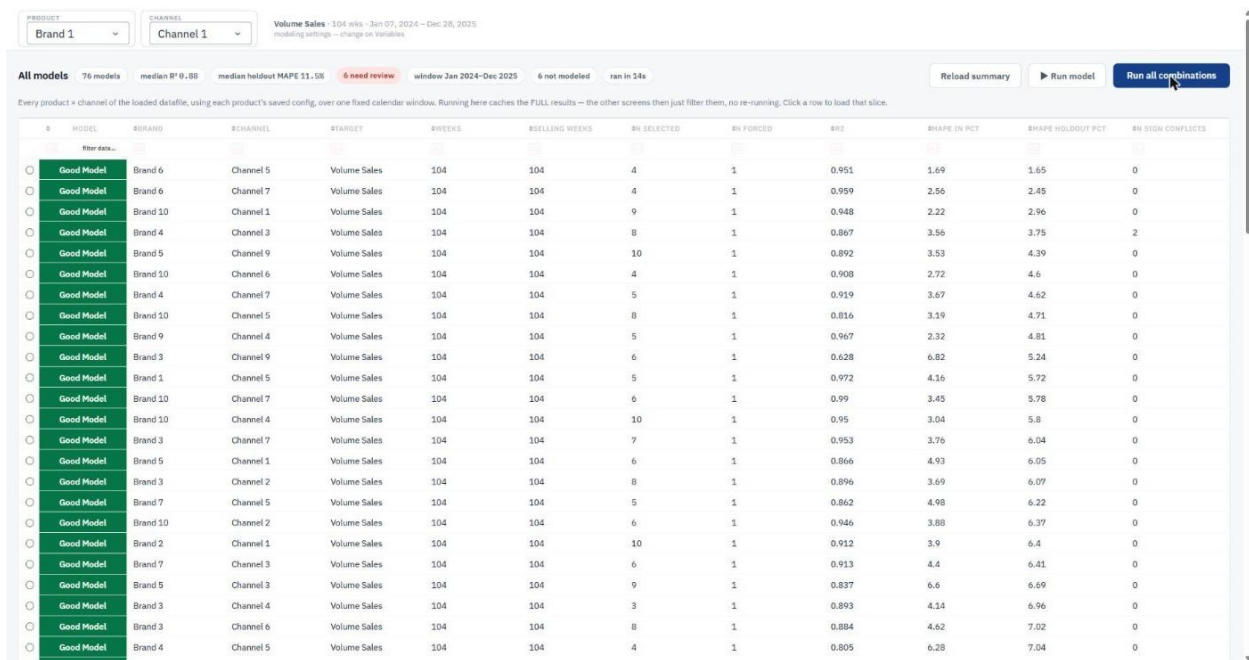
All ten products, all three levels, scored on the same held-out last 13 weeks per channel by WMAPE. Lower is better.

There is no universal answer, which is itself the finding. Unpooled wins on seven products, hierarchical on three, pooled on none outright. At the individual channel level it is more mixed still: of 76 channels, unpooled wins 48, hierarchical 16 and pooled 12. Pooling tends to help thin channels that were fitting noise on their own and to cost the large channel that had plenty of data to speak for itself.

The practical advice: run all three, look at the per-channel table, and use the level that wins for the channels you are being paid to forecast.

WHY WMAPE HERE All three levels are scored by WMAPE, not MAPE. Across channels that differ thirty-fold in size, plain MAPE lets the smallest channel decide the winner — a 200% error on a tiny channel counts exactly as much as a 5% error on the one that is half the product.

10. Model Runs — running everything at once



Model Runs — 76 models in 14 seconds, median R^2 0.88, median holdout MAPE 11.5%, six flagged for review and six not modelled.

Every Product $\times$ Channel in the workbook, one row each, using each product's saved configuration over the one fixed window.

Control	What it does
Run all combinations	Fits everything and caches the full results. About 20 seconds for 76 models. This is the button that makes every other screen fast.
Run model	Re-runs just the currently selected slice.
Reload summary	Re-reads the summary from disk without re-fitting.
Clicking a row	Opens that slice on Diagnostics.
Sort / filter	Standard on every column. Sorting by holdout MAPE descending puts your problems at the top.

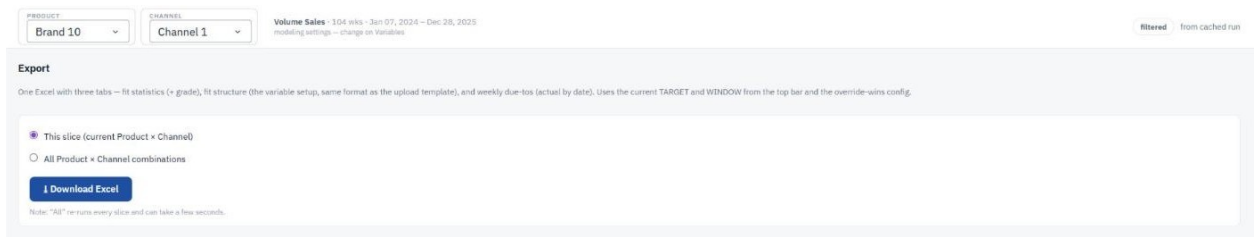
The grade column is a plain-language read on holdout MAPE — Good, Moderate or Bad — so a non-technical reader can scan the table without knowing what MAPE is.

The "Not modelled" table

Underneath is a list of slices the engine deliberately refused to model, each with a reason. A slice needs at least 52 weeks of data and 26 weeks with non-zero sales inside the window. On the current dataset six slices are skipped, including one that previously produced a model with 254% MAPE off 24 selling weeks.

WHY IT IS SHOWN This table exists because silence is the dangerous outcome. A slice with no data in the window should not produce a model — but it must not vanish without comment either, or nobody notices the underlying data problem.

11. Export — getting it into Excel



Export — one slice or all combinations, always using the current target, window and override-wins config.

One workbook, several sheets:

Sheet	What is in it
fit_statistics	One row per model: R^2 , MAPE, holdout MAPE, grade, variable counts, window dates.
fit_structure	The variable setup for every model — the same format as the upload template, plus each variable's current coefficient and whether it made the last model.
weekly_due_tos	Every driver's contribution for every week, with model year, period, quarter and calendar year columns already filled in so it pivots straight away.
due_to_change	Period-over-period due-tos.
errors	Slices that were skipped, with reasons.

Choose "This slice" or "All combinations". The export always uses the current target and window and the override-wins configuration, so what you download matches what you were just looking at.

PIVOT-READY The weekly sheet is laid out for pivot tables on purpose. Alex found the window bug by pivoting this export in Excel, and his advice to the team afterwards was that pivot tables are how the client will interrogate these numbers.

12. Definitions — the glossary inside the app

Every term the tool uses, defined, on a screen inside the tool. It covers the metrics, the roles, the transforms, the selection settings and all three modelling levels — including the reasoning behind the choices, not just the definitions.

PRODUCT

Brand 10

CHANNEL

Channel 1

Volume Sales

104 wks - Jan 07, 2024 - Dec 26, 2025

modeling settings - change on Variables

Filtered from cached run

Definitions & Methods

What the numbers mean and how a model is built. Definitions track the engine's actual behavior.

Fit metrics

R² / adjusted R² — share of weekly variance the model explains; adjusted R² penalizes adding predictors. Shown together so a high R² from over-fitting is visible.

In-sample MAPE — mean absolute % error on the training window, over non-zero-actual weeks only.

Holdout MAPE — out-of-sample error on a reserved tail. The last up to 13 weeks are ALWAYS reserved (independent of the window, as long as ≥5 training weeks remain); a validation model — same structure, selected without seeing the tail — is scored there, and that number is kept as the reported holdout.

Durbin-Watson — residual autocorrelation check (~2 ≈ no autocorrelation).

T-stat — from an unconstrained OLS as an inference reference; |t| ≥ 2 ≈ significant at 5%.

VIF — variance-inflation factor (collinearity), computed on the final in-model variables. > 10 flags redundancy — a non-blocking warning; forced variables can be high by design.

Model grade (Model Runs tab)

Good — holdout MAPE ≤ 10%

Moderate — > 10% and ≤ 40%

Bad — > 40%

No holdout — the slice has no holdout period to score.

These cutoffs are a team convention (reusing the review-flag thresholds) — confirm with the client.

Contribution vs due-to — not the same number

Contribution — a **level**: how much volume a driver accounts for IN a period ($\sum \beta_i \cdot x_i$ over that period's weeks). The additive decomposition of the fitted line — intercept as its own line plus coefficient × value per driver — so intercept + $\sum (\beta_i \cdot x_i)$ reproduces the fitted value exactly each week.

Due-to — a **change**: how much of the movement between TWO periods a driver explains (its contribution in period B minus its contribution in period A). This is the "how is the change in my macro variables affecting my sales year over year" number. The due-to's sum exactly to the change in modeled volume between the two periods.

Periods of unequal length — compared as **per-week averages** automatically — otherwise a 52-week period always looks bigger than a 4-week one for reasons that have nothing to do with the drivers.

Where to find each — the Contributions screen has a toggle: Contribution (level in a period) or Due-to (change between the two selected periods). The export carries both — weekly_due_tos (per-week levels, with period columns) and due_to_change (period-over-period changes).

Avg weekly due-to — the mean of a driver's weekly contribution. For **media** (any variable with an adstock decay) the average is taken **only over weeks with real execution** (non-zero raw spend/impressions) — never diluted by the zero weeks of

Definitions — the same reasoning as this guide, kept beside the numbers it describes.

It is deliberately kept in the app rather than only in this document, because the person who needs a definition needs it while looking at the number, not while looking for a Word file.

13. How do I...? (recipes)

Run everything from scratch

- Load data (bottom left).
- Model Runs → Run all combinations.
- Sort by holdout MAPE, worst first. That is your work queue.

Force a variable into a model

- Variables → find the row → set Role to force.
- Save config, then Run.
- It will now appear on Diagnostics marked as forced, and it is protected from both VIF pruning and the family caps.

Stop a variable being used at all

- Variables → set Role to exclude → Save config → Run.

Change a coefficient bound

- Variables → set Coef lower and/or Coef upper, in the variable's original units.
- These override the sign setting. If you also set a scale, the bound is transformed with it, so a scale can never silently tighten a bound you wrote.

Make a coefficient readable ("per 1,000 impressions")

- Variables → set Scale to 1000 → Run.
- The coefficient becomes a thousand times larger and every contribution, due-to and fitted value stays bit-for-bit identical. It is a units change only.

Apply the same setup to fifty variables

- Variables → Download template.
- Edit the CSV in Excel.
- Upload template. Either the whole file applies or none of it does.

Find out whether I am overspending on a medium

- Saturation → pick the media variable.
- Set a midpoint and slope you believe (or use the optimizer's candidates).
- Read the crossover on the ROI curve. Below 100% means over-invested; above means under-invested.
- If it says "no crossover", your slope is below 1 — see §8.

Justify a decay rate to a client

- Saturation → drag the decay slider.
- The correlation table shows $|r|$ with the target at the model's setting and at yours. That is the evidence.
- Then Apply to config and Run to see what it does to holdout error.

See how a whole brand is performing

- High Level → Total Brand → pick the product.
- Read WMAPE, not MAPE, at this level.
- Check the member grid underneath before quoting the total.

Get one coefficient for a brand instead of nine

- Multi-Level → select the product in the top bar → Run all three levels.
- Read the Pooled column. Check the per-channel fit table to see which channels the shared coefficients serve badly.

Decide whether to pool

- Multi-Level → Run all three levels.
- Look at the per-channel comparison table; the winning cell in each row is highlighted.
- Look at the λ curve. Rising left-to-right means pool; falling means do not.

Reproduce a number in Alex's spreadsheet

- cd code && python test_multilevel.py
- The first block reproduces Hierarchical Modeling Explanation.xlsx column by column.

14. Reading the numbers

Term	What it means
R²	Share of week-to-week variation explained. In-sample, so treat with care.
MAPE	Mean absolute percentage error — the average of each week's percentage error. Every week counts equally regardless of size.
WMAPE	Total absolute error ÷ total actual. Weights by the volume at stake. The right metric for totals and for pooled models.
Holdout MAPE	Error on the last 13 weeks, which variable selection never saw. The honest measure of whether the model predicts.
Bias	Whether the model runs high or low overall. Near zero is expected in-sample; a large bias on a total points at a coverage problem.
VIF	Variance inflation factor. How much a predictor duplicates the others. Above 10 is flagged.
Durbin-Watson	Residual autocorrelation. Near 2 is healthy; near 0 means a time pattern is missing from the model.
Contribution	Volume a driver accounts for during a period — a level.
Due-to	Change in a driver's contribution between two periods — a delta. What "why am I down" asks for.
Impact / SD	Coefficient × standard deviation. Volume per one standard-deviation move. The comparable number across drivers.
Adstock decay	Fraction of this week's execution that carries into later weeks.
Half saturation (γ)	Execution level at which a variable delivers half of everything it can deliver.
Hill slope (s)	Curve shape. Above 1 S-shaped; below 1 concave from the origin.
Average return	Margin dollars returned per dollar spent, on average, at a given spend level.
Marginal return	Margin dollars the NEXT dollar returns at that spend level.
Optimal point	Where average and marginal return cross. Past it, extra spend earns less than what you have already spent.
Index (hierarchical)	A channel's coefficient divided by the mean across channels. 1.00 means inherit the product-level effect unchanged.
λ (shrink)	How much of the index to apply. 0 = pooled, 1 = full channel variation.

15. Files the tool reads and writes

Path	What it is
code/dashboard.py	The app. Screens, controls and callbacks.
code/capstone_pipeline.py	The engine — transforms, selection, constrained fit, contributions.
code/multilevel.py	Pooled and hierarchical models.
code/saturation_curves.py	The SatCurve port and the weekly transform diagnostic.
code/rollup.py	Total Brand / Total Channel aggregation.
configs/varconfig_*.csv	One per product (and optionally per Product × Channel). Plain CSV — editable in Excel, diffable in git.
outputs/	Batch summaries, skipped-model lists, experiment results.
docs/	This guide, the maths notes and the meeting action-item logs.
code/test_multilevel.py	Numerical verification, including Alex's spreadsheet.
code/test_dashboard_screens.py	Drives every new screen end to end.

Configurations are CSV on purpose. They can be edited in Excel, mailed to a colleague, and tracked in version control, and nothing about the tool is required to read them.

Where to change things

Alex asked for a note on where changes would be made, so that the code can be picked up after the project. Briefly:

To change...	Go to
How a transform works	capstone_pipeline.adstock` / `.hill` / `.assemble_matrix`
How variables are selected	capstone_pipeline.forward_stepwise` and `prune_by_vif`
How coefficients are constrained	capstone_pipeline._bounds_from_specs`
The national-predictor split	`multilevel.proportionalize_national` — one line
The hierarchy arithmetic	multilevel.coefficient_index` and `cap_index`
The ROI curve	saturation_curves.SatCurve`
What a total sums	rollup.rollup`
A screen or a button	`dashboard.py` — layout near the top, callbacks below

16. When something goes wrong

What you see	What it means and what to do
"No cached results yet"	The screen needs the results cache. Go to Model Runs and click Run all combinations.
A slice missing from the results	It is in the "Not modelled" table with a reason — almost always too few weeks or too few selling weeks inside the window. That is the data, not the tool.
"has no media variable in its model"	No media was selected for that slice, so there is no curve to draw. Force one in on Variables if it should be there.
Holdout MAPE far worse than in-sample	The model has learned the past and cannot predict. Try Punitive strictness or family caps.
A sign conflict flag	The data pushes that coefficient the opposite way from your prior. The constraint is still enforced. Worth understanding before you ignore it — it is often a data problem, like a lost distribution halfway through the year.
Very high VIF	Two predictors are near-duplicates. The model is fine; the individual coefficients are not defensible. Exclude one, or raise strictness.
"no crossover inside 0–300%"	The slope is below 1, so the return lines never meet. See §8.
A total that dips at the left edge	Check the coverage warning — a slice probably starts part-way through the window.
The app will not start	Read the error. If it names a component id, a control was renamed and the startup self-check caught it.

17. How the modelling actually works

Not required reading, but useful when someone asks you to defend a number.

The pipeline, in order

1. Clip to the fixed calendar window — the same dates for every slice.
2. Reserve the last 13 weeks as a validation tail.
3. Transform each predictor: raw $\rightarrow$ adstock $\rightarrow$ scale $\rightarrow$ Hill saturation. Adstock first because carry-over is about WHEN an impression lands; saturation last because diminishing returns apply to accumulated pressure, not to each week's raw spend. Same order as Meta's Robyn and Google's Meridian.
4. Prune predictors above the VIF threshold, protecting forced ones.
5. Forward stepwise selection on the training weeks only, respecting the family caps at entry.
6. Fit by bounded least squares, enforcing every sign and bound.
7. Score that structure on the untouched validation tail — the holdout MAPE.
8. Refit the same structure on the full window. Those are the reported coefficients.
9. Decompose: contribution of driver i in week t = coefficient $\times$ value. Intercept plus the sum of contributions equals the fitted value exactly.

The saturation curve

The curve is the Hill equation: $H(x) = x^s / (x^s + k^s)$, where k is the half-saturation point and s the slope. Alex described it on the 27 July call and half-remembered its origin — "some Bill and this other data scientist created this for the efficacy of drugs, a curve that has a midpoint and a slope". It is A. V. Hill, 1910, fitting oxygen binding to haemoglobin; it became the standard dose-response curve in pharmacology and is now the saturation function in both Robyn and Meridian. Same shape, same two parameters he named.

A useful consequence: the coefficient on a saturated variable is the maximum weekly volume that variable can deliver, because Hill returns values in $[0, 1]$. Contributions stay additive, so the decomposition is unchanged.

Two things that are deliberately NOT done

- No Bayesian sampler. Alex was explicit that this stays an optimised least-squares engine — "we don't need to do Bayesian; that would be a complete recalculation of everything you guys have done on the back end". So partial pooling is done by indexing and an explicit shrinkage dial rather than by a prior and a posterior.
- No min-max normalisation of predictors. For unregularised constrained least squares it is a linear reparameterisation that changes nothing, selection already standardises internally, and inverting every reported number would introduce exactly the risk it was meant to avoid. Comparability is provided by the Impact / SD column instead, which is invariant to units.

Appendix A — What we found

Findings that came out of building this, rather than instructions for using it. They are here because they affect how the outputs should be read.

1. Channel intercepts decide whether pooling works at all

Without a per-channel intercept, pooled and hierarchical models are badly beaten by unpooled on every product. With one, hierarchical wins outright on three of ten and the chosen λ reaches 1 on four. Brand 3's pooled holdout error went from 39.2% to 13.6%, its hierarchical from 36.9% to 7.2%. Alex's indexed method was not the problem; the specification around it was.

2. Full channel variation is rarely optimal, but often close

Fitting λ on the holdout across all ten products gives $\lambda = 1$ four times, $\lambda = 0$ four times, and something in between twice. There is no single right answer, which is why the dial is exposed rather than hard-coded.

3. The index is unstable where channels disagree

On Brand 1, fifteen of sixteen predictors have a mean coefficient across channels smaller than the spread of those coefficients. The index divides by that mean, so it is dividing by something close to zero and the resulting indices run from -7.2 to $+11.9$. This is a property of the normalisation, not noise in one channel, and it is why the cap and the sign guard exist.

4. Equal sums do not mean national

Seasonality_Index sums to exactly 104.000000 in every channel of a 104-week window while differing substantially week to week. The quick sum test flags it as national; it is not. Any future dataset with normalised per-channel indices will hit the same trap.

5. The curve optimizer improves its own score more reliably than the holdout

Across 25 slices it improved its cross-validation score every time it fired (-2.07 points) but the untouched holdout only half the time (mean $+0.85$ points — slightly worse). It declines to change anything on ten of fourteen media slices. Best case Brand 4 $\times$ Channel 2, 9.84% $\rightarrow$ 6.90%; worst Brand 1 $\times$ Channel 1, 19.64% $\rightarrow$ 25.51%. Media is a small share of these models and 104 weeks is thin for two extra non-linear parameters per variable — which is exactly why Robyn and Meridian use Bayesian priors and geographic pooling here. It therefore ships switched off, as an opt-in that writes candidates rather than results.

6. Fewer variables is sometimes better

Capping Trade at one and Macro at one on Brand 1 $\times$ Channel 1 improved holdout MAPE from 23.0% to 21.5%. The default is no caps, but they are worth trying on any model whose holdout is much worse than its in-sample fit.

Appendix B — How this was verified

Two test files, both runnable from the code folder.

```
cd code
python test_multilevel.py      # 37 numerical checks
python test_dashboard_screens.py # 23 end-to-end screen checks
```

What test_multilevel.py checks

- Alex's Hierarchical Modeling Explanation.xlsx reproduced column by column — the index (F), the final coefficient (G), the capped index (H) and the capped final coefficient (I), plus the sheet's own checks in rows 20–25 that the index averages to 1 and the uncapped coefficients average back to the pooled ones. Agreement to $1e-12$.
- Alex's SatCurve reproduced against an independent recomputation of the Hill curve: the 0–300% grid, the half-saturation formula, the anchoring at 100% spend, and the crossover rule.
- That margin cannot move the optimal point.
- That a concave curve (slope below 1) has no crossover — checked against the closed form marginal $\div$ average = $n \cdot k^n / (x^n + k^n)$, which is always below 1 when $n \leq 1$.
- That the weekly diagnostic plots the exact column the model used, not a re-derivation.
- That the national split conserves national totals (error $2.4e-16$).
- That Seasonality_Index is correctly NOT treated as national.
- That intercept + $\Sigma \beta x$ equals the fitted line for pooled and for every hierarchical channel.
- That each channel's fitted total equals its actual total once channel intercepts are on.
- That channel-level due-tos sum to the pooled totals.
- That $\lambda = 0$ reproduces the pooled coefficients exactly, and $\lambda = 1$ with no cap reproduces Alex's uncapped Model 3.
- That no channel coefficient flips sign against the pooled one with the sign guard on.

What test_dashboard_screens.py checks

Dash callbacks are ordinary functions, so the tests call them with the same arguments the browser would supply and assert that a real component tree comes back rather than an error banner. It covers all three rollup modes, the media picker, the slider defaults, the weekly panel, the correlation table, the auto-derived spend and price, the ROI curve, both multi-level paths, and that each of the nine screens shows exactly one panel.

The app also self-checks its own wiring at startup, walking the layout and confirming every control referenced by a callback exists. That check found one real defect while this release was being built.

What is not covered

There is no browser-level test — nothing clicks buttons in a real page. The callbacks are exercised directly, which catches logic and data errors but not CSS or layout problems. Those are caught by looking at the screen.